

Practica 3: Algoritmo genético.

Luis Fernando Torres Hernández

9 de octubre de 2026

Algoritmo genético.

1. Introducción

En esta practica se desarrollo un algoritmo genético para optimizar el diseño de armaduras planas usando un valor de peso como referencia.

En esta practica se hace un uso intensivo de apuntadores y memoria dinámica, durante la ejecución se mantiene en memoria un arreglo de cromosomas que representan posibles soluciones al problema.

2. Desarrollo

El código se dividio en 3 archivos de código fuente, el primero es `main.c` que contiene la función principal y el flujo de ejecución del programa, el segundo es `comun.c` que contiene funciones que son usadas en varias partes del código y el tercero es `comun.h` que contiene las definiciones de estructuras y prototipos de funciones que se usan en `main.c` y `comun.c`.

Se utilizaron 2 estructuras, la primera es `instancia` que contiene los datos de la instancia del problema, estos son la probabilidad de mutación, la probabilidad de cruce, el peso máximo permitido, el numero de elementos de los que se puede formar la armadura y un arreglo de elementos que contiene el area y longitud de cada elemento.

La segunda estructura es `cromosoma` que contiene un genotipo que es un número entero de 64 bits que representa la selección de elementos que forman la armadura, el peso de la armadura y la aptitud que es un valor que indica que tan buena es la solución representada por el cromosoma, este valor se calcula como

$$\frac{1}{1000 * \|M - P\|}$$

donde M es el peso máximo permitido y P es el peso de la armadura representada por el cromosoma. El código se puede ver en la Figura 1.

```
1  typedef struct instancia{
2      float p_cruza;
3      float p_mutacion;
4      float maxima;
5      int n_objetos;
6      float *area;
7      float *longitud;
8  } instancia;
9
10 typedef struct cromosoma{
11     long long int genotipo;
12     float peso;
13     float aptitud;
14 } cromosoma;
```

Figura 1: Definición de estructuras para la instancia y el cromosoma

2.1. Algoritmo genético

El algoritmo genético que se implemento en esta practica sigue los siguientes pasos:

- Se genera una población inicial aleatoria.
- Se calcula la aptitud de cada cromosoma en la población.
- Se hace una selección de la siguiente generación de cromosomas.
- Se realiza el cruce de los cromosomas seleccionados.
- Se realiza la mutación de los cromosomas seleccionados.
- Se repiten los pasos 2 a 5 por un número determinado de iteraciones.

2.1.1. Selección

La selección de los cromosomas se realizo usando el método de ruleta, este consiste en asignas a cada cromosoma un rango de valores proporcional a su aptitud, luego se genera un número aleatorio y se selecciona el cromosoma cuyo rango contiene ese número, este proceso se repite hasta seleccionar el número de cromosomas deseado lo que se muestra en la Figura 2.

```
1  cromosoma *seleccion(cromosoma *poblacion, int n) {
2      cromosoma *seleccionado = malloc(sizeof(cromosoma)*n);
3      float total_aptitud = 0.0;
4      for (int i = 0; i < n; i++) {
5          total_aptitud += poblacion[i].aptitud;
6      }
7
8      for (int i = 0; i < n; i++) {
9          float r = ((float)rand() / RAND_MAX) * total_aptitud;
10         float acumulado = 0.0;
11         for (int j = 0; j < n; j++) {
12             acumulado += poblacion[j].aptitud;
13             if (acumulado >= r) {
14                 copiar_cromosoma(&poblacion[j], &seleccionado[i]);
15                 break;
16             }
17         }
18     }
19     return seleccionado;
20 }
```

Figura 2: Selección de cromosomas usando el método de ruleta

2.1.2. Cruce

La cruza se realizo usando el método de punto de cruce, este consiste en seleccionar un punto aleatorio en el genotipo y luego intercambiar los bits de los cromosomas seleccionados a partir de ese punto usando una probabilidad de cruce, esto se muestra en la Figura 3.

```
void cruzar(cromosoma *padre1, cromosoma *padre2, float pc) {
    int punto_cruce = rand() % MAX_BITS;
    long long int mascara = (1LL << punto_cruce);
    while (mascara) {
        float probabilidad = (float)rand() / RAND_MAX;
        if (probabilidad < pc) {
            long long int bit1 = padre1->genotipo & mascara;
            long long int bit2 = padre2->genotipo & mascara;
            padre1->genotipo = (padre1->genotipo & ~mascara) | bit2
                ;
            padre2->genotipo = (padre2->genotipo & ~mascara) | bit1
                ;
        }
        mascara >>= 1;
    }
}
```

Figura 3: Cruce de cromosomas usando el método de punto de cruce

2.1.3. Mutación

La mutación se realizo bajo el siguiente criterio, se genera un número aleatorio y si este es menor a la probabilidad de mutación se invierte el valor del bit correspondiente en el genotipo del cromosoma, esto se muestra en la Figura 4.

```
void mutar(cromosoma *poblacion, int n, float pm) {
    for (int i = 0; i < n; i++) {
        long long int mascara = 1;
        for (int j = 0; j < MAX_BITS; j++) {
            float probabilidad = (float)rand() / RAND_MAX;
            if (probabilidad < pm) {
                poblacion[i].genotipo ^= mascara; // Cambia el bit
                    correspondiente
            }
            mascara <<= 1;
        }
    }
}
```

Figura 4: Mutación de cromosomas usando el método de inversión de bits

El programa permite seleccionar un archivo de datos que serán cargados para hacer la optimización de la armadura y ejecuta el algoritmo genético por

un número determinado de iteraciones, la ejecución se puede ver en la Figura 5.

```
Iteración 10:
Población:
No.  | Bits                                                                | Peso    | Aptitud
-----+--------------------------------------------------------------------+---------+------------
[ 1] | 00110100001110110010010111100110000010111111111000101110011111 | 4110.48 | 0.000000677506762
[ 2] | 01110100110100011111101001010111001110100000011010001111O0111 | 3000.60 | 0.000000386847205
[ 3] | 11001110000001111111011000110011000101001111110011000011111001 | 4498.56 | 0.000000919117667
[ 4] | 11001100111101101111100000101010110011000101100011101010011110 | 3110.76 | 0.000000404040406
[ 5] | 11110011010000010100100000011000100001111011101100001010111001 | 4957.20 | 0.000001589825160
[ 6] | 11111000010001010101100000011010110000110011100111001010101101 | 2034.72 | 0.000000281610824
[ 7] | 10101001110000010101100110011001110001111011100100001101000l | 3812.40 | 0.000000563697881
[ 8] | 01110101111101101111101111000111101101111110001010010010111 | 5721.48 | 0.000007462686426
[ 9] | 10011010010010111110000011010000111011000000100110010110011O | 1837.44 | 0.000000266737800
[10] | 11001110111001110111111100010111010001001001110010101011111111 | 2797.56 | 0.000000358551461
[11] | 10110100010100010011111000111101010010110011100011010101001 | 2506.32 | 0.000000324675312
[12] | 10011010000101110100110010011011000100000010110001100010000l0 | 3179.52 | 0.000000415454934
[13] | 01111010111001110101011010011001001000111101100011000111010 | 2862.00 | 0.000000367107191
[14] | 01101110000111011101001010101001110101011111111100001010101l | 2995.20 | 0.000000385951381
[15] | 11100001110000010100100010011000100001000101111011000100010l1 | 3584.52 | 0.000000499500516
[16] | 01111010110010111110001010000100110111111011100110000011011l | 3752.28 | 0.000000545256285
[17] | 10101001111010010100110110001000010001101101110000100000110101 | 3780.72 | 0.000000554016594
[18] | 11000010100000010100000011010010100001111001100110001010101l | 3112.92 | 0.000000404367171
[19] | 11110100111101111101000111111101001001100010101100110111110 | 2327.40 | 0.000000306842594
[20] | 01101110000111011001001010001000000000101100100101011111010 | 2720.16 | 0.000000348918348
Mejor cromosoma hasta ahora: Genotipo = 01110101111101101111101111000111101101111110001010010010111l, Peso = 5721.48, Aptitud = 0.000007462686426
Peso máximo permitido: 5586.59
```

Figura 5: Ejecución del programa